



SPARK Framework: From MVP Concept to Execution

Practical Planning & Real-World Application

A practical guide to planning and managing your
Midterm MVP.

*MVP: Minimum Viable Product

Why Adopt the SPARK Framework?

01

The "Coder's Trap"

- No plan = messy code
- Leads to delays and rewrites
- SPARK forces you to plan first

02

The Logic of Execution

- Answers 5 key questions:
- Why? What? Who? When? What if?
- Covers all project bases

03

Future-Proofing

- Same logic used in startups
- Useful for research grants in lab
- Learn once, use forever

S - Scope & Story

Motivation (Pain Point)

- What exact problem does the user have?

Target Audience

- Who needs this? Be very specific.
- (e.g., "Junior CS students", not "everyone")

MVP Core Value

- What is the ONE most important feature?

⚠ Out-of-Scope (Crucial)

- List what you are NOT building.
- Prevents the project from getting too big.

Examples:

- "We will NOT build a mobile app."
- "We will NOT add real-time video."

P - Plan & Product

System Architecture

- Keep it simple: Frontend ↔ Backend ↔ DB
- Show how parts connect

Tech Stack Choices

- Why did you choose this?
- Good reasons: "Team knows it", "Fits the need", "Easy to deploy"

Feature Breakdown

- Separate your features into two lists.

1. Must-Have (MVP):

- Features required for the midterm.
- Must be completed on time.

2. Nice-to-Have (Future):

- Features for later.
- Prevents the project from failing.

A - Allocation

Roles

- Be specific about who does what.
- Avoid vague titles like "Dev".

Example:

Backend Dev: Handles Database & API
DevOps: Handles AWS Deployment

Schedule

- Break work into 2-week blocks.
- Each block needs a real result.

Timeline:

W2: Design done
W4: Core works
W6: Deployed
W8: Live Demo

Checkpoints

- Set dates to check progress.
- Adjust the plan if needed.

Key Checks:

W1: Rules set
W3: APIs fixed
W5: Testing
W8: Ready

R - Risk

Technical Risks

- API goes over the free limit.
- System crashes when too many people use it.
- Database gets too slow.

Schedule Risks

- Midterm exams stop the work.
- Other classes have big deadlines.

Solutions (Plan B)

- Risk A: Use a simpler, free method.
- Risk B: Remove the hardest feature.
- Risk C: Finish the core part early.

Rule of Thumb

- Always know what you will cut if you run out of time.

K - Key Metrics

Done (DoD)

- System runs 24/7.
- Answers in under 3 seconds.
- Code is on GitHub.
- Tested by 10 real users.

Numbers

- 50 active users in Week 1.
- 100 successful uses.
- 99% uptime.

Future Value

- Make it open source for others.
- Write a blog post about it.
- Show it on your resume.

The "Code-Switching" Matrix

SPARK	Academic Grant	Startup Pitch	Student Project
S - Scope	Research Problem	Market Need	User Pain Point
P - Plan	Methodology	Product Roadmap	Architecture
A - Allocation	Team Expertise	Go-to-Market	Who Does What
R - Risk	Uncertainties	Competitors	Tech Limits
K - Metrics	Publications	Revenue	User Numbers

Conclusion: Learn SPARK now, and you can use it anywhere later.

Case Study: CampusLink

Project Name

- "CampusLink" - AI Study Group Discord Bot

The Concept

- Help CS students find study buddies quickly.
- Use a Discord bot to match them automatically.

Why This Example?

- It is simple and easy to understand.
- It perfectly shows all 5 parts of the SPARK framework.

CampusLink: S (Scope & Story)

Pain Point

- CS students struggle to find study buddies.
- Facebook/Dcard are too slow.

Target User

- Year 1 & 2 CS students at our school.

MVP Core Value

- Auto-match students in the same class.
- Create a private Discord chat instantly.

⚠ Out-of-Scope

- We will NOT build a mobile app. (Using Discord instead)
- We will NOT build a chat system from scratch.
- We will NOT match students from other schools yet.

CampusLink: P (Plan & Product)

Discord API ↔ Node.js Server ↔ PostgreSQL DB

Tech Stack

- Backend: Node.js
All team members know JS.
- Database: PostgreSQL
Good for relating users to courses.
- Deployment: AWS
Using the free tier.

Feature List

Must-Have (Midterm):

- Student ID login
- Input course name
- Basic matching
- Auto-create Discord channel

Future (Finals):

- AI matching by coding languages
- Find common free time

CampusLink: A (Allocation)

Team Roles (3 Members)

- **Alice (PM/QA):**
Manages schedule, writes tests.
- **Bob (Database/DevOps):**
Designs database, writes queries.
Handles Docker and AWS.
- **Charlie (API):**
Connects Discord, writes match logic.

Milestones

- **Week 2:**
Architecture and Database design done.
- **Week 4:**
Bot runs locally, basic functions work.
- **Week 6:**
API connected, deployed to AWS.
- **Week 8 (Midterm):**
Live demo with 10 real users.

CampusLink: R (Risk)

Technical Risks

- OpenAI API might cost too much money.
- Discord might block the bot if too many students use it at once.

Schedule Risks

- Midterm exams in Week 7 will stop all work.

Solutions (Plan B)

- Fallback to simple keyword string-matching (SQL LIKE queries) instead of using LLM.
- Implement a queue system for Discord API calls.
- Cut the "Auto-creation of Discord channels" feature. Instead, just send a direct message (DM) to users with their matched partner's Discord ID.

CampusLink: K (Key Metrics)

Done (DoD)

- Bot runs 24/7 on AWS.
- Matches users in under 3 seconds.
- All code is on GitHub.
- Tested by 10 CS students.

Numbers

- Get 50 active users in Week 1.
- Process 100 successful matches.
- Keep the bot online 95% of the time.

Future Value

- Share code with other university clubs.
- Write a blog post about how we built it.
- Show off our full-stack skills for jobs.

Call to Action

Your Task

Submit a "SPARK Project Plan" for your Midterm MVP.

How We Grade

- Scope: Is your MVP realistic? Did you list what you won't do?
- Tech: Do your tech choices make sense?
- Time: Are your milestones doable? Do you have a Plan B?

Format Rules

- Use the template provided.
- Use bullet points. Keep it clean.
- You can draw diagrams by hand.

Next Steps

1. Form your team.
2. Find a real problem to solve.
3. Fill out the SPARK Project Plan.
4. Get ready for a 5-minute pitch.